

## Dinamički izrazi svojstava

sVB podržava korišćenje niza kao dinamičkog objekta, kome možete dodati dinamička svojstva.

Postoje dva različita načina da se to uradi:

### 1. Korišćenje konvencije imenovanja podataka

Možete dodati reč Data – *podaci* kao prefiks ili sufiks bilo kom identifikatoru da biste ga tretirali kao dinamički objekat, tako da mu možete dodati svojstva koristeći tačkastu notaciju kao da je u pitanju normalan izraz svojstva. Na primer, možete kreirati objekat Car – *automobil* koji ima svojstva Color i Speed ovako:

```
CarData.Color = Colors.Red  
CarData.Speed = 100  
TextWindow.WriteLine({CarData.Speed, CarData.Color })
```

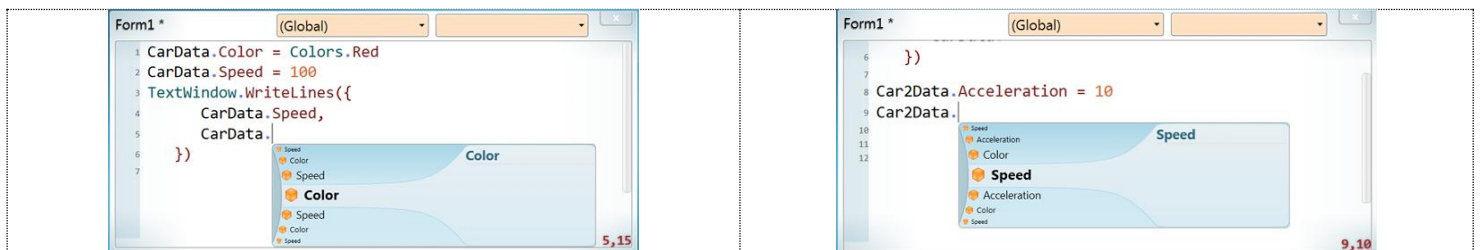
U stvari, sVB konvertuje gornju sintaksu u:

```
CarData["Color"] = Colors.Red  
CarData["Speed"] = 100  
TextWindow.WriteLine({CarData["Speed"], CarData["Color"] })
```

Možete koristiti oba, ali korišćenje dinamičkog objekta ima sledeće prednosti:

a. Kraći je i čitljiviji.

b. IntelliSense nudi automatsko dopunjavanje imena dinamičkih svojstava, što vas sprečava da ih pogrešno kucate, kao što bi se moglo desiti prilikom pisanja ključeva niza stringova bez ikakve podrške za editor (*slika dole levo*).



c. Dinamički objekat može da nasleđuje imena svojstava od drugog dinamičkog objekta ako sadrži njegovo ime (nakon skraćivanja dela „Data“ iz oba). Na primer, objekti Car2Data i myCarData će prikazivati svojstva Color i Speed (nasleđena od objekta CarData) u svojoj listi za automatsko dovršavanje (*slika gore desno*).

Takođe, objekat MyCar2Data će naslediti sva svojstva od MyCarData, Car2Data i CarData!

### 2. Korišćenje operatora pretrage rečnika

Dodavanje reči Data imenima promenljivih čini ih dužim i možda vam se neće svideti njihovo korišćenje, pa vam sVB nudi alternativni način definisanja dinamičkih svojstava, korišćenjem operatora ! (koji se u VB.NET-u naziva operator pretrage rečnika). Dakle, primer CarData može se jednostavno prepisati kao:

```
Car!Color = Colors.Red  
Car!Speed = 100  
TextWindow.WriteLine({Car!Speed, Car!Color})  
Car2!Acceleration = 10  
Car2!Speed = 200
```

Ovo je lepo kraći kod, ali može vam trebati vremena da se naviknete na pisanje ! umesto tačke.

## Domeni dinamičkih svojstava

Prilikom nasleđivanja imena dinamičkih svojstava, sVB ignoriše pravila domena promenljivih, kako bi vam omogućio da ponovo koristite svojstva u potprogramama i funkcijama.

Ovo je potpuno bezbedno jer utiče samo na imena svojstava koja će se pojaviti u listi automatskog dovršavanja, ali nema uticaja na vrednosti promenljivih u vreme izvršavanja.

To znači da lista automatskog dovršavanja može da vam ponudi imena svojstava iz dinamičkog objekta iz druge funkcije, ali ako pročitate vrednosti ovih svojstava u kodu, oni će vratiti prazne stringove bez obzira na njihove vrednosti u drugoj funkciji i neće imati nikakve vrednosti osim ako ih ne podesite u trenutnoj funkciji. Dakle, dva dinamička objekta su i dalje dve različite lokalne promenljive, uprkos tome što koriste slična imena ključeva!

Na primer, kada pokrenete ovaj kôd, svojstvo Car!Color u potprogrami Car1 će prikazati vrednost, dok će isto svojstvo u potprogrami Car2 prikazati praznu vrednost:

```
Car1()  
Car2()  
Sub Car1()  
car!Color = Colors.Red  
car!Speed = 100  
TextWindow.WriteLine({"car1 speed = " + car!Speed, "car1 color = " + car!Color})  
EndSub  
Sub Car2()  
car!Speed = 200  
TextWindow.WriteLine({"car2 speed = " + car!Speed, "car2 color = " + car!Color})  
EndSub
```

```
car1 speed = 100  
car1 color = #FFFF0000  
car2 speed = 200  
car2 color =
```

## Naredbe za dodelu

Naredbe za dodelu koriste operator = da bi postavile levi izraz na vrednost desnog izraza, u obliku:

```
Levi izraz = desni izraz
```

Gde desni izraz može biti bilo koji validan izraz, ali levi izraz može biti samo jedan od ovih izraza:

### 1. Izraz identifikatora:

Leva strana naredbe za dodelu može biti identifikator, koji vam omogućava da definišete nove promenljive ili promenite vrednost postojećih. Na primer:

```
X = 1      ' Deklarišite novu promenljivu X  
Y = X + 2  ' Deklarišite novu promenljivu Y  
X = 0      ' Promenite vrednost X
```

sVB može da zaključi tip promenljive iz njene početne vrednosti, tako da su X i Y zaključeni kao Double u gornjem primeru.

Imajte na umu da se ime promenljive ne može pojaviti na obe strane u njenoj deklaracijskoj liniji. Ovaj kôd će vam dati grešku pri kompajlaciji:

```
Z = Z + 1
```

Ali je u redu koristiti ime promenljive na obe strane nakon što je inicijalizovana. Ovaj kôd će se pravilno kompajlirati:

```
Z = 2  
Z = Z + 1
```

Poslednji red može biti zbunjujući, jer izgleda kao nemoguća matematička jednačina, jer Z

nikada ne može biti jednako  $z+1$ !

Zabuna je uzrokovana zato što se simbol `=` koristi u BASIC jezicima za obavljanje dve funkcije: dodeljivanje i logičko poređenje, a ovde ga koristimo za prvu.

Da biste razumeli kako funkcioniše, trebalo bi da znate da će se desna strana prvo izračunati, a zatim će vrednost biti dodeljena levoj strani. Dakle, gornji kôd možete pročitati kao:

Nova vrednost Z biće jednaka staroj vrednosti Z plus 1

A tokom izvršavanja, kod će izgledati ovako nakon zamene z sa 2:

```
Z = 2 + 1
```

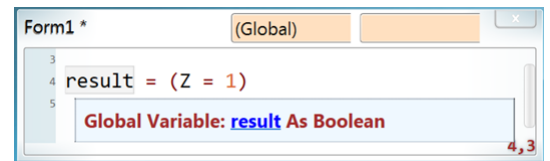
Pogledajte i ovu izjavu:

```
rezultat = Z = 1
```

Ovde imamo dva simbola `=` u istoj izjavi. Po pravilu, samo prvi može biti operator dodele, tako da sledeći mora biti operator jednakosti. Dakle, gornja izjava može se prepisati kao:

```
rezultat = (Z = 1)
```

Gde je izraz sa desne strane logički izraz koji će vratiti True ili False, i zato sVB zaključuje da je rezultujuća promenljiva tipa Bulova vrednost:



Takođe možete kreirati niz dodeljivanjem inicijalizatora niza njegovom identifikatoru:

```
a = {1, 2, 3}
```

I možete kopirati gornji niz u drugi:

```
b = a
```

Mala osnovna promena koja može dovesti do preokreta: ovaj smešan kôd se kompajlira u SB:

```
x = y  
y = 1
```

Gde se y smatra deklarisanim jer je dodeljeno u drugom redu, a njegova vrednost će biti "" u prvom redu!

Ovo se više neće kompajlirati u sVB, jer vam ne dozvoljava da pročitate promenljivu pre nego što je inicijalizujete.

## 2. Izraz niza

Leva strana naredbe dodele može biti izraz niza, koji vam omogućava da definišete nove nizove ili promenite vrednost postojećih. Na primer:

```
A[1] = 1 ' Definišite niz i postavite prvu stavku  
A[2] = 2 ' Postavite drugu stavku  
A[1] = 1 ' Menja se prva stavka  
A[3] = A[1] + A[2]  
B[1][1] = 5
```

## 3. Izraz svojstva

Naredbu dodele možete koristiti da postavite vrednost svojstva ovako:

```
TextBox1.Text = "Zdravo sVB"
```

## 4. Dinamički izraz svojstva

Leva strana naredbe dodele može biti dinamički izraz svojstva, koji vam omogućava da definišete nova dinamička svojstva ili promenite vrednost postojećih. Na primer:

```
P!X = 0  
P!Y = P!X + 1  
P!X = 100
```

## Komentari u dokumentaciji

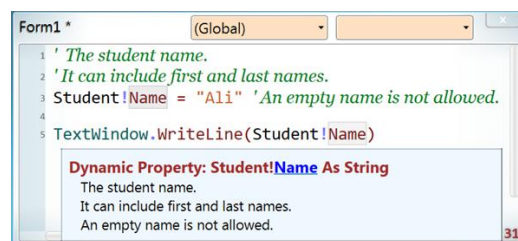
sVB intellisense će koristiti svaki komentar koji navedete na kraju promenljive linije kao informacije o dokumentaciji, koje će biti prikazane na listi za automatsko dovršavanje i u iskaćućem prozoru pomoći (*slika dole levo*):

Isto važi i za deklaraciju dinamičke osobine (*slika dole desno*):



Na gornjoj slici vidite da iskaćući prozor pomoći prikazuje komentar za dinamičko svojstvo ID, a njegovo ime se pojavljuje kao hiperlink, tako da možete kliknuti na njega da biste prešli na red deklaracije.

Takođe možete dati dodatne redove za komentare dokumentacije iznad promenljive ili reda deklaracije dinamičkog svojstva, a ako dodate komentar na kraj reda deklaracije, on će im biti dodat:



## Globalne i lokalne promenljive

Small Basic nema opsege promenljivih, jer se sve promenljive smatraju globalnim i možete ih definisati na bilo kom mestu u datoteci (unutar ili izvan potprograma) i koristiti ih sa bilo kog drugog mesta u datoteci (gore ili dole).

Small Visual Basic je sredio ovaj nered, što je promena koja može sprečiti pokretanje nekih potprograma, verovatno u sVB-u, ali je neophodan korak da bi deca i početnici organizovali svoj kôd i pisali čiste kodove.

Ovo je takođe neophodno da bi parametri potprograma i funkcija ispravno funkcionisali i da bi vam se omogućilo korišćenje rekurzivnih potprograma i funkcija.

Ovo su nova pravila opsega koja primenjuje sVB:

- Promenljive definisane izvan potprograma i funkcija su globalne promenljive.
- Parametri potprograma i funkcija su lokalni i skrivaju sve globalne promenljive sa istim imenima.

- Brojač (iterator) petlje For u bilo kojem potprogramu ili funkciji je lokalni i skriva sve globalne promenljive sa istim imenom.

- Promenljiva definisana unutar bilo kog potprograma ili funkcije je lokalna, osim ako ne postoji globalna promenljiva sa istim imenom definisanim iznad ovog potprograma ili funkcije. Ali ako je globalna promenljiva definisana ispod, onda će je lokalna promenljiva sakriti.

Zato, kao dobru praksu, sledite ove smernice:

- Definišite sve globalne promenljive na samom vrhu datoteke.
- Dajte globalnim promenljivim prefiks (kao što je „g\_“) da biste izbegli eventualne sukobe sa lokalnim promenljivim.

- Ne koristite globalne promenljive osim ako ne postoji drugo rešenje.

Umesto toga, prosleđujte vrednosti potprogramima i funkcijama preko parametara i primajte

vrednosti od funkcija preko njihovih povratnih vrednosti.

## Naredbe Label i Goto

Naredba Label je veoma jednostavna. To je samo identifikator praćen dvotačkom:

```
Label1:
```

a funkcija ifs je jednostavnija, jer samo označava liniju koja vam omogućava da skočite pomoću GoTo naredbe:

```
Goto Label1
```

U stvari, Goto naredba je manje važna u sVB-u nego SB, jer sVB ima sigurnije komande za skok kao što su Return, ExitLoop i ContinueLoop, dok je GoTo komanda jedini način za takve skokove u SB! Dakle, nikada ne smete koristiti GoTo u sVB-u!

### Napomene:

1. Labela je lokalna za potprogram ili funkciju u kojoj je definisana. To znači da ne možete skočiti iz bilo kog potprograma ili funkcije.

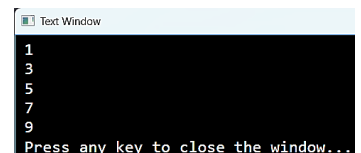
2. Možete definisati labelu u globalnom delu datoteke (van potprograma i funkcije), ali i dalje ne možete skočiti na njih iz potprograma niti funkcije.

3. Ne možete definisati lokalnu labelu sa istim imenom kao globalna labela!

4. Ne skačite u sredinu for petlje, u suprotnom ćete dobiti neočekivane rezultate.

5. Možete skakati gore ili dole, ali budite oprezni, jer skakanje gore može dovesti do toga da program uđe u beskonačnu petlju, osim ako ne navedete uslov za izlazak. Na primer, ovaj kôd koristi GoTo za izvršavanje for petlje:

```
Start = 1
_To = 10
_Step = 2
I = Start
_Next:
If I >= _To Then
Goto _ExitLoop
EndIf
TextWindow.WriteLine(I)
I = I + _Step
Goto _Next
_ExitLoop:
```



U stvari, sVB kompajler prevodi For petlju u kod sličan gore navedenom (sa nekim dodatnim kodom za rukovanje negativnim koracima u petljama).

Dakle, Goto je instrukcija niskog nivoa i bolje je da je ostavite kompajleru i pokušate da je ne koristite sami osim ako ne morate.